Last week:

- Bridges (edges that affect connectedness).
- Trees (connected graphs with no cycles).
  - Examples (stars, double stars)
  - Leaves exist. Pruning leaves allows induction within trees.
  - $|E| = |V| - 1$

This week:

- Spanning trees.
- Counting trees.

Recall that a subgraph H of a graph G is spanning if $V(H) = V(G)$.

Definition. *A **spanning tree** of a connected graph G is a spanning subgraph $H \subseteq G$ that is also a tree.*

Example (Two spanning trees in a graph).



Theorem (4.10). *If G is a connected graph, then G contains a spanning tree.*

$|E(G)| \geq n-1$

Idea: We can create a spanning tree by deleting edges which don't affect connectedness, eventually we will land with $|E(T)| = n-1$

Proof: Let $\mathcal{F} = \{$all spanning graphs of G which are connected$\}$

We have $G \in \mathcal{F}$, so $\mathcal{F}$ is not empty.

Take $T \in \mathcal{F}$ with minimum number of edges.

Claim: T is a spanning tree

We know T is connected and spanning ($T \in \mathcal{F}$)

For a contradiction assume T contains a cycle C

Let e denote an edge in C.

Consider $T' = T - e$

- T' is spanning
- T' is connected (e is <u>not</u> a bridge in T)
- No cycles in T'

$\Big\} \Rightarrow T' \in \mathcal{F}$

But $|E(T')| = |E(T)| - 1$, contradicts minimality of T.

□

Exercise (HW 5.2 Alternative proof of the previous theorem)**.** *Let* G *be a connected graph. Consider the family* $\mathcal{F}$ *of all spanning subgraphs of* G *that contain no cycle. Show that* $\mathcal{F}$ *is non empty and that an element of* $\mathcal{F}$ *with* ***maximum*** *number of edges is a spanning tree of* G*.*

Exercise (A short proof for Theorem 4.7)**.** *Use Theorem 4.10 to prove that if* G *is a connected graph, then*

$$|E(G)| \geqslant |V(G)| - 1.$$

Proof: Let T spanning tree, $T \subseteq G$

$$|E(G)| \geq |E(T)| = |V(T)| - 1 = |V(G)| - 1$$

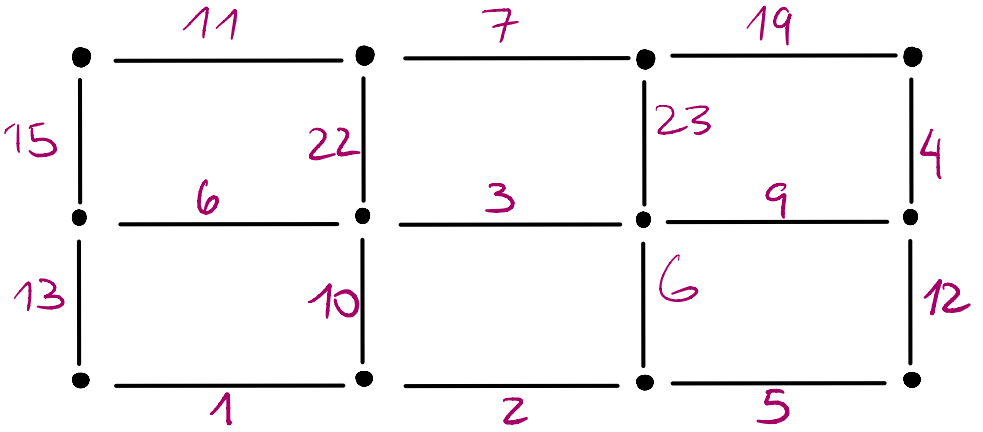
Note: A spanning tree constitutes a minimal set of paths in a connected graph (if we discard any edge in the tree, it disconnects.) Sometimes some edges are more *expensive* than others.

Definition. *Fix a connected graph* G.

- *We say that* G *is a* ***weighted graph*** *if it comes with a* ***weight function*** $w\colon E(G) \to \mathbb{R}^{>0}$.
- *The* ***weight*** *of a subgraph* H *of* G *is defined as* $w(H) = \sum_{e \in E(H)} w(e)$.
- *A* ***minimum spanning tree (mst)*** *of* G *is is spanning tree* T *such that* $w(T)$ *is minimum among all spanning trees.*

→ These exist!

Example (A weighted graph)**.**



Algorithm: (Kruskal, 1956) G weighted

START with a forest
grow T by adding edges

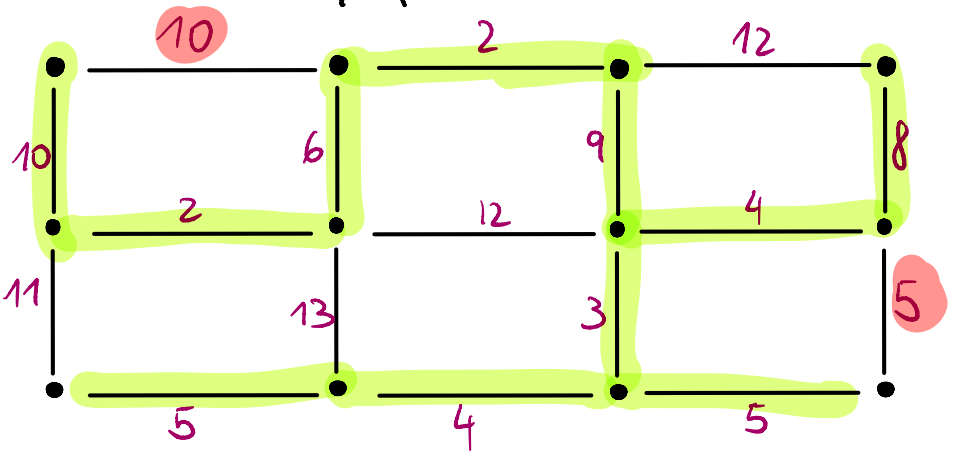
(1) Order $E(G)$ by increasing weight according to w.

(2) Start with T being the empty spanning subgraph of G.

(3) Add edges one-by-one according to the ordering if the edge creates no cycle.

(4) End if adding the next edge (5) would create a cycle.

any of



$w(T) = 58$

2 2
2 +2
3 +3
4 +4
4 +4
5 +5
5
5
6
8
9
10
10
11
12
13

20
5
6
8
9
10
38

58

Start with a tree
Grow by adding vertices

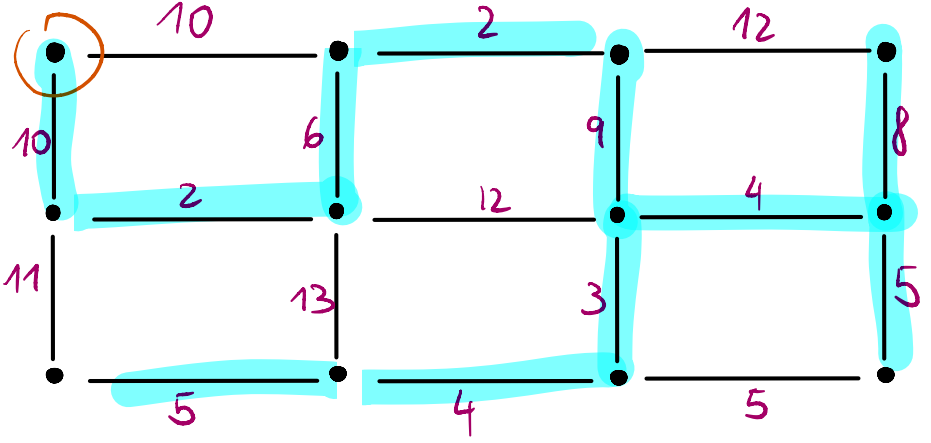
Algorithm: (Prim–Jarník, 1957–1930)

(1) Start with T being a single vertex in G.

(2) Find an edge e of smallest weight that has only one endpoint in T. Add e to T.

(3) End when every edge has two endpoints in T.

of G



$w(T) = 59$

Exercise. *For the following graph, use the algorithms above to construct minimal spanning trees..*

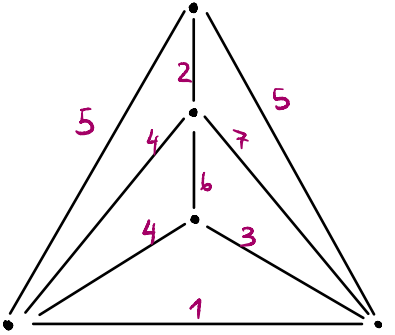
Algorithm: (Kruskal, 1956)

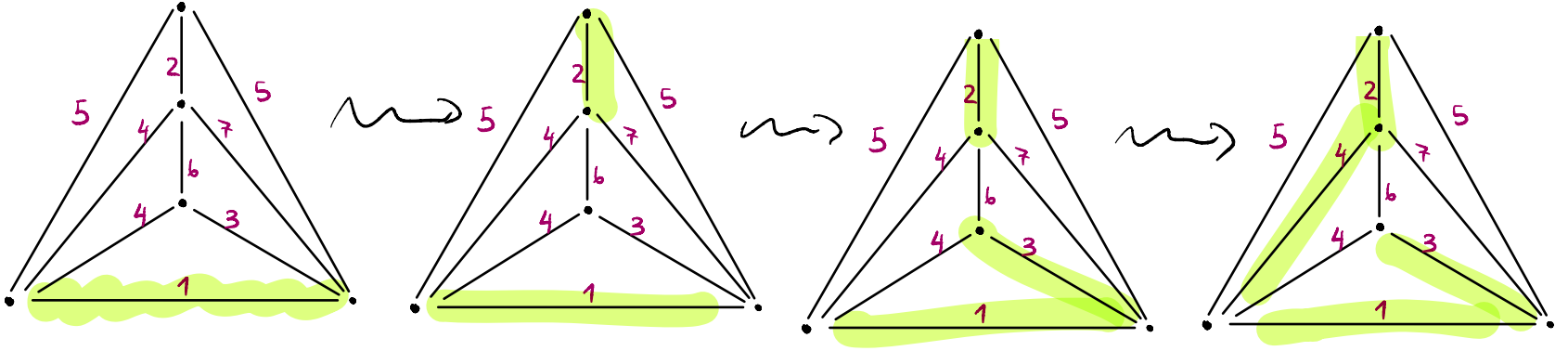
(1) Order $E(G)$ by increasing weight according to w.

(2) Start with T being the empty spanning subgraph of G.

(3) Add edges one-by-one according to the ordering if the edge creates no cycle.

(4) End if adding the next edge (any of 5's) would create a cycle.





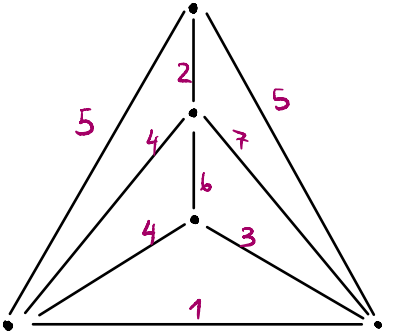
Exercise. *For the following graph, use the algorithms above to construct minimal spanning trees..*

Algorithm: (Prim–Jarník, 1957–1930)

(1) Start with T being a single vertex in G.

(2) Find an edge e of smallest weight that has only one endpoint in T. Add e to T.

(3) End when every edge has two endpoints in T.



Algorithm: (Kruskal, 1956)

(1) Order $E(G)$ by increasing weight according to w.
(2) Start with T being the empty spanning subgraph of G.
(3) Add edges one-by-one according to the ordering if the edge creates no cycle.
(4) End if adding the next edge(s) would create a cycle.

any of

Theorem (4.11). *Kruskal's algorithm produces a minimum spanning tree in a connected weighted graph.*

Proof: Fix weighted graph G. And let T denote the output.

1st: T contains no cycles by construction.

2nd: T is connected. If T contains two components, there is an edge in G connecting these components. The algorithm would use that edge, because it creates no cycles in T.

3rd: T is spanning: if a vertex of G is not used in T, then the algorithm would use some incident edge.

Thus T is a spanning tree.

Assume G has n vertices.

Let $e_1, \dots, e_{n-1}$ denote the edges of T, chosen by Kruskal in order

Thus $w(e_1) \le w(e_2) \le \dots \le w(e_{n-1})$, and $w(T) = \sum_{i=1}^{n-1} w(e_i)$

Assume for a contradiction T not minimum.

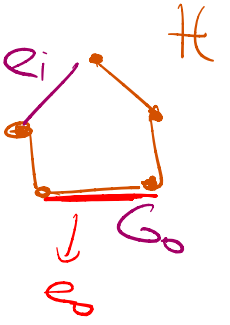
Let $\mathcal{F} = \{$minimum spanning trees of $G\}$

Let $H \in \mathcal{F}$ with maximum number of edges in common with T.

Let H share $e_1, \dots, e_{i-1}$ with T, but not e_i

Consider $G_0 = H + e_i$

- G_0 contains a cycle C using e_i (H is spanning)
- $\exists$ edge $e_0 \in E(C) \setminus E(T)$ (T contains no cycles)



Consider $T_0 = G_0 - e_0$. Then T_0 is a spanning tree

- T_0 connected (G_0 connected, $e_0 \in G_0$ not a bridge)
- $|E(T_0)| = |E(G_0)| - 1 = |E(H)| + 1 - 1 = |E(H)| = n-1$

Thus T_0 spanning tree

But: 1) T_0 has more edges in common with T than H does

$$T_0 = H + e_i - e_0$$

e_i → in T, e_0 → not in T

2) $w(T_0) = w(H) + w(e_i) - w(e_0)$

3) $w(H) \leq w(T_0)$ because H MST.

} $w(e_0) \leq w(e_i)$ (*)

We would contradict the definition of H if T_0 MST

Note H has $e_1, \dots, e_{i-1}, e_0$

T has $e_1, \dots, e_{i-1}, e_i$

} $w(e_0) \geq w(e_i)$ (**)

Because of Kruskal

Now (*) + (**) $w(e_0) = w(e_i)$

Thus by (2) $w(T_0) = w(H)$, i.e. To MST with more edges in common with T Than H with T

∎

Theorem (4.12). *Prim's algorithm produces a minimum spanning tree in a connected weighted graph.*

Theorem. *If all edge weights in a connected graph* G *are distinct, then* G *has a unique minimum spanning tree.*

There is a third algorithm that can be run in parallel